

EXERCÍCIOS DE FIXAÇÃO:

Arquivo Base (index.html)

Crie este arquivo HTML. Ele contém um formulário de cadastro com campos para Nome, E-mail, Senha e Confirmação de Senha, além de elementos de erro ocultos.

HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Cadastro - Validação Progressiva</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: center; height:
100vh; background-color: #f4f4f9; }
    form { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 4px 8px rgba(0,0,0,0.1);
display: flex; flex-direction: column; gap: 10px; width: 300px; }
    input { padding: 10px; border: 1px solid #ccc; border-radius: 4px; }
    button { padding: 10px; background-color: #28a745; color: white; border: none; border-radius: 4px;
cursor: pointer; }
    .erro-msg { color: red; font-size: 12px; display: none; }
    .input-erro { border-color: red; } /* Classe para estilizar o erro conforme FAQ */
  </style>
</head>
<body>
  <form id="meuFormulario">
    <h3>Crie sua Conta</h3>

    <input type="text" id="nome" placeholder="Seu nome completo">
    <span id="erroNome" class="erro-msg">Por favor, preencha seu nome.</span>

    <input type="email" id="email" placeholder="Seu e-mail">
    <span id="erroEmail" class="erro-msg">Formato de e-mail inválido.</span>

    <input type="password" id="senha" placeholder="Sua senha (mín. 6 caracteres)">
    <span id="erroSenha" class="erro-msg">A senha precisa ter pelo menos 6 caracteres.</span>

    <input type="password" id="confirmaSenha" placeholder="Confirme sua senha">
    <span id="erroConfirmaSenha" class="erro-msg">As senhas não coincidem.</span>

    <button type="submit">Cadastrar</button>
  </form>
  <script src="script.js"></script>
</body>
</html>
```

Os 10 Exercícios Progressivos (Aplicados no script.js)

Crie um arquivo `script.js` para responder aos exercícios, vamos construí-lo gradativamente.

Nº 1 - Título: Escutando o Formulário

Descrição: Para iniciar a manipulação do DOM sem poluir o HTML com atributos `onclick`, você deve selecionar o formulário pelo seu ID e adicionar um "ouvinte" para o momento em que o usuário tentar enviá-lo.

Entrega: Código JavaScript contendo a seleção do elemento `form` via `querySelector` e o método `addEventListener` escutando o evento `submit`.

Nº 2 - Título: Assumindo o Controle (Prevenção de Recarregamento)

Descrição: O comportamento padrão do navegador ao enviar um formulário é recarregar a página (o que atrapalha a validação *client-side*). Modifique a função do ouvinte de eventos criado no exercício 1 para interceptar e cancelar essa ação padrão.

Entrega: Adição do parâmetro `event` na função do ouvinte e a execução do comando `event.preventDefault()`; logo na primeira linha.

Nº 3 - Título: Captura de Dados e Variável de Controle

Descrição: Agora que o recarregamento foi impedido, você precisa "pegar" o que o usuário digitou e preparar o terreno para as validações. Crie também uma variável de controle para registrar se o formulário como um todo é válido ou não.

Entrega: Variáveis em JavaScript utilizando `.value` para armazenar os dados dos campos (Nome, E-mail, Senha, Confirmação) e uma variável `let formValido = true;`.

Nº 4 - Título: Validação Simples (Campo Vazio)

Descrição: Crie sua primeira regra de negócio. O campo "Nome" não pode ser enviado em branco ou apenas com espaços. Se estiver vazio, a mensagem de erro oculta no HTML deve aparecer e o formulário deve ser marcado como inválido.

Entrega: Uma estrutura condicional (`if/else`) que utiliza o método `.trim() === ""` no valor do nome. Se verdadeiro, altera o `display` do erro correspondente para `block` e muda a variável `formValido` para `false`.

Nº 5 - Título: O Molde da RegEx para E-mail

Descrição: Expressões Regulares são padrões poderosos para validar texto. Antes de validar o e-mail preenchido, você precisa definir no código qual é o formato aceitável (algo que contenha texto, "@", texto, "." e texto).

Entrega: Declaração de uma constante `regexEmail` armazenando o padrão literal `/^[^\s@]+@[^\s@]+\.[^\s@]+$/;`.

Nº 6 - Título: Testando o E-mail contra a RegEx

Descrição: Utilize a Expressão Regular criada no exercício anterior para testar o valor que o usuário digitou no campo de e-mail. Caso o teste falhe, exiba a mensagem de erro de formato inválido.

Entrega: Condicional `if` utilizando o método `!regexEmail.test(emailValor)`. Em caso de falha, exibe a mensagem de erro de e-mail e define `formValido = false`.

Nº 7 - Título: Segurança Básica: Comprimento de Senha

Descrição: Para garantir um mínimo de segurança no cadastro (visando a integridade dos dados no banco), crie uma validação que impeça o envio caso a senha escolhida tenha menos de 6 caracteres.

Entrega: Condicional verificando a propriedade `.length < 6` da string da senha. Se for menor, exibe a mensagem de erro específica de senha e invalida o formulário.

Nº 8 - Título: Validação Lógica (Igualdade de Senhas)

Descrição: Um erro comum do usuário é errar a digitação da senha. Verifique se o conteúdo digitado no campo "Senha" é exatamente igual ao digitado no campo "Confirmar Senha". Se não for, bloqueie o envio.

Entrega: Condicional `if` comparando os dois valores (`senhaValor !== confirmaSenhaValor`). Exibição do erro de senhas divergentes e alteração de `formValido` para `false` caso não coincidam.

Nº 9 - Título: Feedback Visual Avançado (Uso de Classes)

Descrição: Como visto no fechamento da aula, usar estilos diretos (inline) não é a melhor prática. Refatore o feedback visual do campo E-mail para, em vez de apenas mostrar a mensagem, adicionar uma classe CSS específica (`input-erro`) ao campo `input` quando ele for inválido, e removê-la quando for corrigido.

Entrega: Atualização da validação do Exercício 6 incluindo os comandos

`inputEmail.classList.add('input-erro')` (no caso de erro) e `inputEmail.classList.remove('input-erro')` (no bloco `else`).

Nº 10 - Título: O Veredito Final (Sucesso e Reset)

Descrição: Após todas as validações, se o usuário preencheu tudo perfeitamente, a variável de controle criada no Exercício 3 terá sobrevivido intacta. Exiba uma mensagem de sucesso ao usuário e, em seguida, limpe todos os dados do formulário para que ele fique pronto para um novo cadastro.

Entrega: Condicional final verificando se `formValido` continua `true`. Se sim, execução de um `alert("Cadastro realizado com sucesso!")` seguido da chamada da função nativa `formulario.reset()`;